

Technology Half-Life: Analysing Temporal Degradation of Answer Freshness on Stack Overflow

Alan Styslavski
TU Delft (5573866)
Delft, Netherlands

Miguel El Khal
TU Delft (5723086)
Delft, Netherlands

Antoni Nowakowski
TU Delft (5454700)
Delft, Netherlands

Abstract

In the rapidly evolving IT landscape, frequent framework and language updates can cause once-accurate technical information to become obsolete within years, or even months. This paper looks into the "half-life" of technical knowledge on Stack Overflow. We use temporal web analytics and sentiment analysis to quantify the rate at which answer utility and relevance degrade over time. Specifically, we compare the utility decay of answers tied to rapidly-evolving (ephemeral) technologies, such as frontend frameworks, against stable foundations like SQL to identify significant differences in their degradation rates.

1 Introduction

The World Wide Web has changed how software developers find and check technical knowledge. Social web platforms allow users to create, rate, and organise content together, and these platforms have become important places for sharing programming knowledge [6]. Stack Overflow is one of the most important examples in software engineering. Its voting and ranking system helps developers quickly find answers to technical problems. As a result, many developers treat highly voted Stack Overflow answers as practical and trustworthy guidance, sometimes even before consulting official documentation.

However, the authority of Stack Overflow also creates a problem. Software changes continuously. Frameworks release new versions, APIs become deprecated, and common programming practices are replaced by newer approaches [4, 14]. Stack Overflow signals such as votes and accepted answers show how useful an answer was at the time it was written, but they do not clearly show when that answer has become outdated. This creates an important Web Science problem. The Web is very good at storing and ranking knowledge, but it does not always ensure that this knowledge remains valid over time.

Previous research shows that this problem can mislead developers, with [15] showing that obsolete Stack Overflow answers can lead developers toward incorrect or outdated solutions, especially in fast changing technology areas. However, much of this work treats obsolescence as something that is either present or absent. Less attention has been given to obsolescence as a gradual process. It is still unclear whether answers about fast changing technologies lose usefulness more quickly than answers about stable topics, or how quickly this decline happens.

This study addresses that gap by using the concept of technology half life. In this paper, technology half life refers to the estimated time after which a Stack Overflow answer has lost a substantial part of its practical usefulness. Building on Munger's [9] view of temporal validity as a measurable property, we treat obsolescence as a rate of change rather than a fixed state. We expect this rate to differ

between fast changing technologies, such as frontend frameworks, and more stable foundations, such as SQL. Section 2 explains these concepts and categories in more detail.

To measure this decline, we need signals that reflect changes over time, not only past approval. Comments are useful for this purpose because they can be added long after an answer was posted and can directly state whether a solution still works [12]. This leads to a second part of our study: whether comments about freshness can help identify knowledge degradation alongside vote based signals. [2] and [8] show that sentiment and opinion in software engineering text can reflect technical judgement, not only personal preference. This means that comment analysis should be adapted to the software engineering domain, which forms the basis of the method described in Sections 2 and 4. Based on this motivation, this paper addresses the following research questions:

- **RQ1:** How does the utility of Stack Overflow answers change over time for fast changing technologies compared to more stable technologies?
- **RQ2:** To what extent can developer comments serve as temporal signals of answer obsolescence, and which comment cues best indicate knowledge degradation?

This paper makes three main contributions. First, it defines answer utility using community signals from Stack Overflow, such as votes, accepted answers, and comments, and studies how this utility changes over time across different technology categories. Second, it develops a comment based approach for identifying signs of answer freshness or obsolescence, using developer feedback such as comments that mention outdated or deprecated solutions. Third, it compares fast changing and more stable technology categories to show whether technical knowledge becomes outdated at different rates on the Web. These findings can help explain how knowledge platforms might better identify, update, or reduce the visibility of time sensitive answers. The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 describes the dataset. Section 4 explains the methodology. Section 5 presents the evaluation strategy. Section 6 reports the results. Section 7 discusses the findings and limitations. Section 8 concludes the paper.

2 Background and Related Work

2.1 Stack Overflow as a Web Knowledge Platform

Stack Overflow was introduced as a platform in 2008 featuring questions and answers on certain computer programming topics. It has evolved over the years but the main idea has remained the same, users can post a question, people on the platform may provide an answer, and depending on the correctness of the answer, users can upvote or downvote it. Importantly highly voted answers can remain highly visible for years, even after the underlying technology

has changed. This creates a Web Science problem that this paper addresses: the Web preserves and ranks knowledge, but does not always keep it temporally valid.

2.2 Technical Knowledge Obsolescence

Technical Knowledge on the Web is highly time dependent. Programming knowledge is often tied to specific versions of languages, libraries, frameworks, APIs, development tools or even operating systems. An answer posted years ago can be no longer applicable years later for a newer technology that evolved from an older version. This can be due to an API endpoint being removed, a package changing its syntax or many other reasons[4, 14]. For this reason, the usefulness of a technical answer depends on both its original correctness and on whether it remains applicable in the current technical environment.

This paper approaches technical knowledge obsolescence as the process by which a previously useful technical answer loses practical applicability over time. This doesn't mean that the answer was originally wrong, but that the validity of an answer can degrade because the technological context has changed. This creates a specific problem for Web-based technical knowledge platforms. Stack Overflow ranks and preserves answers using community signals such as votes, accepted-answer status, comments, and edits. These signals often reflect usefulness at the time of interaction, but they do not automatically expire when technologies change. As a result, older answers can remain highly visible even after their practical value has declined.

2.3 Obsolete Answers on Stack Overflow

Several studies have examined the quality, reliability, and maintenance of Stack Overflow content. The most directly relevant work is the empirical study by Zhang et al.[15] on obsolete answers on Stack Overflow. That research looked at how answers become obsolete, what characteristics obsolete answer have in general and how the Stack Overflow community deals with that. They found that obsolete answers can mislead answer seekers, especially when the outdated information is not clearly marked or updated. Their findings also show that obsolescence is not evenly distributed across all topics: answers in tags such as node.js, ajax, android, and objective-c were more likely to become obsolete [15].

However, existing work leaves room for further investigation. Prior research has mainly focused on identifying obsolete answers and describing their characteristics. Less attention has been given to modeling answer obsolescence as a measurable decay process over time. In particular, it remains unclear whether answers connected to fast-changing technologies lose utility at a faster rate than answers connected to more stable technologies. This paper addresses that gap by comparing the temporal degradation of answers across technology categories. Rather than only detecting obsolete answers, the study aims to estimate how quickly answer utility declines and whether this decline differs between ephemeral technologies, such as frontend frameworks, and stable foundations, such as SQL or basic algorithms.

Additionally this paper examines whether comment-based signals can be used as indicators of knowledge degradation. While votes and accepted-answer status provide useful information about

how the answer was perceived initially. Comments, by contrast are often added after the original answer and may directly report whether a solution still works. For this reason this paper treats comments not merely as a discussion artifacts but as a possible sign that an answer has lost its freshness. The contribution is to evaluate whether sentiment analysis and freshness-related comment classification can help identify when an answer begins to lose practical utility.

2.4 Answer Utility, Freshness and Knowledge Degradation

To study the degradation of Stack Overflow answers, this paper makes a distinction between three related but separate concepts, namely answer freshness, answer utility and knowledge degradation.

Answer freshness refers to the degree to which an answer remains technically valid and applicable within the current version of the relevant technology. A fresh answer is one whose code, explanation, commands, or recommended practices can still be applied without requiring major changes. Freshness is therefore connected to the present technical environment, not only to the correctness of the answer at the time it was posted.

Answer utility refers to the practical usefulness of an answer for developer attempting to solve a problem. Utility is broader than freshness. An answer may still be useful if it explains an underlying concept clearly, even if some implementation details are outdated. On the other hand an answer can be technically correct but have low utility if it's not understandable or is difficult to apply in practice. It is difficult to measure perfectly so it will be approximated using signals available from Stack Overflow, including including score, accepted-answer status, edits, comment activity, and freshness-related comments.

Finally knowledge degradation refers to the decline in the applicability, correctness or usefulness of a technical knowledge over time. In this context, knowledge degradation occurs when changes in languages, libraries, frameworks, APIs, etc. reduce the usefulness of an existing answer.

These definitions are important because Stack Overflow's ranking mechanisms can preserve historically useful answers even after their current utility has declined. Measures like votes and accepted-answers may only reflect a historical utility or community approval, occasionally accumulated for years. They do not indicate whether an answer remains valid in the present. However, comments and edits may provide more time-sensitive evidence of degradation, especially when users explicitly mention version changes.

The concept of "technology half-life" builds on this distinction[9]. In this paper, half-life refers to the estimated time after which an answer loses a substantial portion of its practical utility. This does not imply that an answer becomes completely useless after a fixed period. Rather, it provides a way to model and compare the rate at which answer usefulness declines across different categories of technology.

2.5 Ephemeral vs Stable Technologies

Ephemeral technologies are defined as technologies that often go through version changes, API revisions or tooling updates in their

lifespan[4]. Examples may include frontend frameworks and rapidly evolving platforms such as React, Angular, Vue.js, Node.js, Android, or iOS development. Answers related to these technologies may become outdated when a new major version is released, when a commonly used method is deprecated, or when the accepted best practice changes.

On the other side of the spectrum we have stable technologies, which are those whose core concepts and usage patterns change more slowly over time. Examples may include SQL, relational database concepts, basic algorithms, data structures, or regular expressions. It's important to note the they still may change over time, however their central ideas usually stay the same and remain applicable over long periods of time.

2.6 Community Feedback as a Signal of Degradation

Stack Overflow answers are surrounded by multiple forms of community feedback, including votes, accepted-answer status, comments, and edits. These signals are important because they shape answer visibility and may also provide evidence about answer quality. Votes are one of the most visible signals of answer quality on Stack Overflow. A high score usually indicates that many users found an answer helpful. However, votes are cumulative and historical. An answer may receive many votes during the period when it is fresh and continue to appear authoritative even after the underlying technology has changed[11]. Accepted-answer status has a similar limitation. The accepted answer reflects the solution that helped the original asker, but it does not necessarily represent the best or most current solution for later users[3, 11]. Edits can provide stronger evidence of maintenance. If an answer is edited after a major technology change, this may indicate that the author or community attempted to preserve its freshness. Edits may update syntax, add warnings, mention newer versions, or remove deprecated recommendations. It's important to note that the edits are not guaranteed. Prior works suggest that few old answers are updated once their applicability decays.

2.7 Sentiment Analysis as a Degradation Indicator

Sentiment analysis is commonly used to identify positive, negative, or neutral opinions in textual data. In the context of Stack Overflow, however, ordinary sentiment polarity is not sufficient on its own. Developer comments often communicate technical validity rather than emotional opinion. For example, a comment such as "this method is deprecated in version 4" may not contain strongly negative emotional language, but it is still a clear signal that the answer may have degraded [2]. Similarly, a comment such as "still works in 2026" may be more important as a freshness signal than as a positive sentiment expression.

The goal is not simply to classify comments as positive or negative, but to identify whether they contain evidence about the current usefulness of an answer. Sentiment analysis should therefore be framed as a supporting technique rather than the entire method. The main analytical focus should be freshness-related comment detection. General sentiment can help identify negative or positive

user reactions, but domain-specific classification is needed to detect whether the comment actually refers to technical obsolescence.

3 Methodology

3.1 Dataset

We used the April 21, 2026 data dump of stackoverflow.com [ref].

From the dump we used four tables: *Posts* (questions and answers, distinguished by *PostTypeId*, with answers carrying a *ParentId* linking to their question and the question carrying an *AcceptedAnswerId*), *Votes* (one dated row per vote event, typed by *VoteTypeId*), *PostHistory* (edit, rollback, and post-notice events with full timestamps), *Tags* (together with a join table *PostTags*), and *Comments*.

Each table in the dump is a flat XML file in which every record is a single row element, with small fields encoded as attributes. Since there is no nested structure to flatten, we ingested the files using streaming XML parsing into a relational MySQL database.

During ingestion we applied targeted reductions. From *Posts* table, we retained only questions (*PostTypeId* = 1), and answers (*PostTypeId* = 2). From *Votes*, we kept only upvotes, downvotes, and events, and bounties (*VoteTypeId* 2, 3, 1, and 8 respectively). We removed *PostHistory*, *Tags*, and *Comments* that did not reference the *Posts* that remained. These filters discard the large majority of irrelevant rows at load time and keep the working dataset within the resources our workstations.

3.2 Defining and Detecting Obsolescence

An answer can be judged obsolete in several ways, and these are not synonyms: downvote accumulation primarily signals that an answer is *wrong*; obsolescence comments signal that it is *deprecated* or *outdated*; being overtaken as the accepted answer by another signals that it has been *superseded* (which can occur without the answer ever becoming wrong or outdated). We make this distinction explicit because the choice of signal partly determines what "decay" means in our results. Each candidate signal below results a datable onset, which what makes it usable as a survival event time.

- **Obsolescence comments:** We used NLP [details todo] on comments under an answer to detect statements that it is outdated; the date of the first such comment is the onset of the question becoming obsolete. This is the most direct signal and follows established practice. It is subject to survivorship bias, since obsolescence comments are sometimes deleted (for example, after the answer's author updates their answer), and deleted answers are absent from the dump.
- **Overtaking:** By analysing acceptance events (*Votes* with *VoteTypeId*=1) tied to answers under the same question post, we can determine when an answer gets overtaken by another. The onset is the date of that overtake.
- **Net vote-velocity sign flip:** We considered date at which the answer's net note-velocity (number of upvotes minus number of downvotes in one day) falls under -5, to be the date the answer stops being useful.
- **Bounty after acceptance:** A bounty start (*VoteTypeId* = 8) placed on a question that already has an accepted answer is a deliberate signal that the standing answer no longer satisfies. The bounty date provides the onset.

For each accepted (or once accepted) answer, we computed the time from the moment the answer became accepted for the first time (first occurrence of *Vote* with *VoteTypeId=1*) until the first symptom (onset) of the question becoming obsolete.

4 Evaluation

4.1 Technology Buckets

To compare answer decay across domains with different rates of change, we partition tags into two buckets. The *ephemeral* bucket comprises tags associated with rapidly-evolving technologies whose ecosystems, APIs, or best practices shift on timescales of months to a few years (e.g. `react`, `angular`, `vue.js`, `webpack`, `docker`, `kubernetes`, `swift`, `typescript`). The *stable* bucket comprises tags associated with technologies whose core semantics change slowly or not at all (e.g. `c`, `c++`, `sql`, `algorithms`, `data-structures`, `regex`, `bash`).

An answer is assigned to a bucket based on the primary tag of its parent question, defined as the first tag in the question's tag list. Questions tagged with technologies that do not fall cleanly into either bucket are excluded from this comparison. [TODO: report final bucket sizes and the fraction of the dataset each represents.]

4.2 Survival Analysis Setup

We model answer obsolescence as a time-to-event outcome and apply the Kaplan-Meier estimator [5] separately to the ephemeral and stable buckets. The survival time for each answer is measured from the moment the answer first became accepted.

Rather than committing to a single obsolescence signal, we use a *composite endpoint*: an answer is considered to have experienced the event at the time of the first signal to fire among the four defined in Section 3 (obsolescence comment, overtaking, net vote-velocity flip, or bounty after acceptance). This treats each signal as an alternative observable symptom of the same underlying process, and avoids the arbitrary choice of one proxy over another. Answers for which no signal fires before the data dump date are right-censored at the elapsed time between first acceptance and the dump date.

The *half-life* of a bucket is defined as the estimated median survival time, i.e. the time $t_{1/2}$ at which the Kaplan-Meier survival function $\hat{S}(t)$ first falls to or below 0.5.

[Rest relies on data]

5 Results

6 Discussion

6.1 Threats to validity

7 Conclusion

References

- [1] Arpit Bhayani. 2025. Why Half Your Skills Expire Every Few Years. <https://arpitbhayani.me/blogs/half-life/>. Accessed: 2026-05-22.
- [2] Fabio Calefato, Filippo Lanubile, Federico Maiorano, and Nicole Novielli. 2018. Sentiment Polarity Detection for Software Development. *Empirical Software Engineering* 23, 3 (2018), 1352–1382. doi:10.1007/s10664-017-9546-9
- [3] Boyan Goda, Pavneet Singh Kochhar, David Lo, and Ferdian Thung. 2015. The Synergy between Voting and Acceptance of Answers on StackOverflow – Or the Lack Thereof. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. doi:10.1109/SANER.2015.7081845
- [4] André Hora, Romain Robbes, Marco Tulio Valente, Nicolas Anquetil, Anne Etien, and Stéphane Ducasse. 2018. How Do Developers React to API Evolution? A Large-Scale Empirical Study. *Software Quality Journal* 26, 1 (2018), 161–191. doi:10.1007/s11219-016-9344-4
- [5] E. L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481. <http://www.jstor.org/stable/2281868>
- [6] Won Kim, Ok-Ran Jeong, and Sang-Won Lee. 2010. On Social Web Sites. *Information Systems* 35, 2 (2010), 215–236. doi:10.1016/j.is.2009.08.003
- [7] Shamanth Kumar, Fred Morstatter, and Huan Liu. 2014. *Twitter Data Analytics*. Springer. doi:10.1007/978-1-4614-9372-3
- [8] Bing Liu. 2012. *Sentiment Analysis and Opinion Mining*. Synthesis Lectures on Human Language Technologies, Vol. 5. Morgan & Claypool Publishers. <https://www.cs.uic.edu/~liub/FBS/SentimentAnalysis-and-OpinionMining.pdf>
- [9] Kevin Munger. 2023. Temporal Validity as Meta-Science. *Research & Politics* 10, 3 (2023). doi:10.1177/20531680231187271
- [10] Felipe Ortega, Gregorio Convertino, Massimo Zancanaro, and Tiziano Piccardi. 2014. Assessing the Performance of Question-and-Answer Communities Using Survival Analysis. arXiv:1407.5903 [cs.HC] doi:10.48550/arXiv.1407.5903
- [11] Denis Parra, Alexandros Karatzoglou, Emine Yilmaz, and Mesut Yilmaz. 2017. The Myopia of Crowds: Cognitive Load and Collective Evaluation of Answers on Stack Exchange. *PLOS ONE* 12, 3 (2017). doi:10.1371/journal.pone.0173396
- [12] Shrey Sengupta and Caroline Haythornthwaite. 2020. Learning with Comments: An Analysis of Comments and Community on Stack Overflow. *Proceedings of the Association for Information Science and Technology* 57, 1 (2020). doi:10.1002/pr2.234
- [13] Mohammadreza Tavakoli, Maliheh Izadi, and Abbas Heydarnoori. 2020. Improving Quality of a Post's Set of Answers in Stack Overflow. In *Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 512–519. doi:10.1109/SEAA51224.2020.00084
- [14] Jerin Yasmin, Yuan Tian, and Jinqui Yang. 2020. A First Look at the Deprecation of RESTful APIs: An Empirical Study. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. <https://jinquiayang.github.io/papers/icsme20-webapi.pdf>
- [15] Haoxiang Zhang, Shaowei Wang, Tse-Hsun Chen, Ying Zou, and Ahmed E. Hassan. 2021. An Empirical Study of Obsolete Answers on Stack Overflow. *IEEE Transactions on Software Engineering* 47, 4 (2021), 850–862. doi:10.1109/TSE.2019.2906315